

# Experiment Report

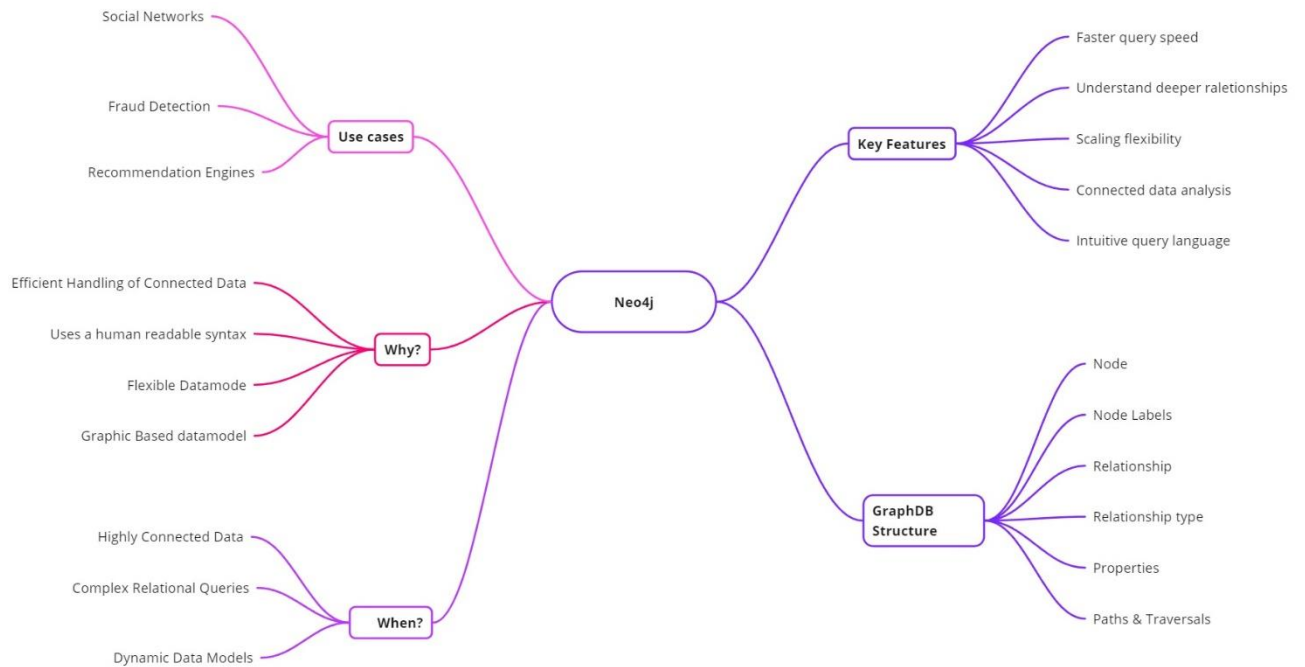
**Student:** Denisenko Sofiia 狄苏菲

**Student id:** 1820249051

**Course:** Big Data Analysis Technology

**Experiment Topic:** Graph DB and Neo4j Assignment

## Mind Map

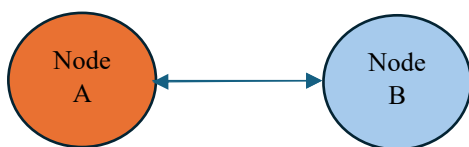


## Task

Explain native graph store and non-native graph store, compare the difference. Explain why the native graph store query more efficient than non-native graph store by using diagram based on the storage mechanism?

**Answer:** Native graph store specifically designed for graphs, it handles high-speed queries and scales well with highly connected data. Non-native graph store built on general purpose databases (relational), so the graph data is disconnected and have a slower query performance.

Native Graph Store:



Relationship

Non-native Graph Store:



(Requires Index Lookup)

Native Graph Store is more efficient due to its optimized processing mechanism, enabling rapid access and traversal with no need to look through indexes.

## Experiment

### Basic Neo4j Usage

neo4j\$ RETURN 'Neo4j is working' AS result

Table

Text

result

1"Neo4j is working"

Creating nodes and relationship, querying the graph.

neo4j\$ CREATE (a:Person {name: 'Alice', age: 30}) CREATE (b:Person {name: 'Bob', age: 25})

Table

Added 2 labels, created 2 nodes, set 4 properties, completed after 216 ms.

neo4j\$ CREATE (a)-[:FRIEND\_OF]→(b)

Table

Created 2 nodes, created 1 relationship, completed after 71 ms.

| a.name  | a.age | b.name | b.age |
|---------|-------|--------|-------|
| "Alice" | 30    | "Bob"  | 25    |

## Simple part

### 1. Create People Nodes

```
CREATE (alice:Person {name: 'Alice', age: 30}),
      (bob:Person {name: 'Bob', age: 25}),
      (charlie:Person {name: 'Charlie', age: 35}),
      (david:Person {name: 'David', age: 28});
```

neo4j\$ CREATE (alice:Person {name: 'Alice', age: 30}), (bob:Person {name: 'Bob', age: 25})

SUCCESS

Added 4 labels, created 4 nodes, set 8 properties, completed after 13 ms.

## 2. Create Interests Nodes

```
8 CREATE (basketball:Interest {name: 'Basketball'}),
9      (music:Interest {name: 'Music'}),
10     (photography:Interest {name: 'Photography'});
11
```

```
neo4j$ CREATE (basketball:Interest {name: 'Basketball'}), (music:Interest {name: 'Music'}), (photography:Interest {name: 'Photography'})
```

**SUCCESS**

Added 3 labels, created 3 nodes, set 3 properties, completed after 58 ms.

## 3. Create Friendships between People

```
MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'}),
      (c:Person {name: 'Charlie'}), (d:Person {name: 'David'})
CREATE (a)-[:FRIEND_OF]→(b),
      (b)-[:FRIEND_OF]→(c),
      (a)-[:FRIEND_OF]→(d),
      (c)-[:FRIEND_OF]→(d);
```

```
neo4j$ // Step 3: Create Friendships between People MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'}), (c:Person {name: 'Charlie'}), (d:Person {name: 'David'})
```



Created 4 relationships, completed after 179 ms.

## 4. Connect People with their Interests

```
MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'}),
      (c:Person {name: 'Charlie'}), (d:Person {name: 'David'}),
      (basketball:Interest {name: 'Basketball'}), (music:Interest {name: 'Music'}), (photography:Interest {name: 'Photography'})
CREATE (a)-[:INTERESTED_IN]→(basketball),
      (b)-[:INTERESTED_IN]→(music),
      (c)-[:INTERESTED_IN]→(photography),
      (d)-[:INTERESTED_IN]→(music);
```

```
neo4j$ // Step 4: Connect People with their Interests MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'}), (c:Person {name: 'Charlie'}), (d:Person {name: 'David'}), (basketball:Interest {name: 'Basketball'}), (music:Interest {name: 'Music'}), (photography:Interest {name: 'Photography'})
```



Created 4 relationships, completed after 144 ms.

## 5. Query for Mutual Friends between Alice and Charlie

// Step 5: Query for Mutual Friends between Alice and Charlie

```
MATCH (a:Person {name: 'Alice'})-[:FRIEND_OF]→(mutual:Person)←[:FRIEND_OF]-(c:Person {name: 'Charlie'})
RETURN 'Mutual friends between Alice and Charlie:' AS Description, mutual.name AS MutualFriend;
```

```
neo4j$ MATCH (a:Person {name: 'Alice'})-[:FRIEND_OF]→(mutual:Person)←[:FRIEND_OF]-(c:Person {name: 'Charlie'}) RETURN 'Mutual friends between Alice and Charlie:' AS Description, mutual.name AS MutualFriend;
```



Description

MutualFriend

"Mutual friends between Alice and Charlie:"

"David"

Started streaming 1 records after 36 ms and completed after 49 ms.

## 6. Find People Interested in Music

```
MATCH (p:Person)-[:INTERESTED_IN]→(i:Interest {name: 'Music'})
RETURN 'People interested in Music:' AS Description, p.name AS Person;
```

```
neo4j$ MATCH (p:Person)-[:INTERESTED_IN]-(i:Interest {name: 'Music'}) RETURN 'People interested in Music:' AS Description...
```

|   | Description                   | Person  |
|---|-------------------------------|---------|
| 1 | "People interested in Music:" | "Bob"   |
| 2 | "People interested in Music:" | "David" |

Started streaming 2 records after 37 ms and completed after 41 ms.

## 7. Suggest Friends based on Shared Interests

```
MATCH (p1:Person)-[:INTERESTED_IN]-(i:Interest)-[:INTERESTED_IN]-(p2:Person)
WHERE NOT (p1)-[:FRIEND_OF]-(p2)
RETURN 'Suggested friends based on common interest:' AS Description, p1.name AS Person1, p2.name AS Person2, i.name AS SharedInterest;
```

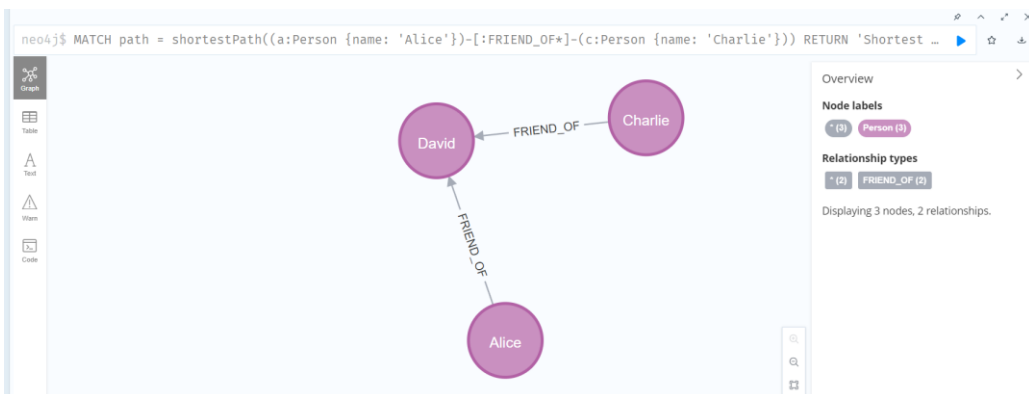
```
neo4j$ MATCH (p1:Person)-[:INTERESTED_IN]-(i:Interest)-[:INTERESTED_IN]-(p2:Person) WHERE NOT (p1)-[:FRIEND_OF]-(p2) RE...
```

|   | Description                                   | Person1 | Person2 | SharedInterest |
|---|-----------------------------------------------|---------|---------|----------------|
| 1 | "Suggested friends based on common interest:" | "David" | "Bob"   | "Music"        |
| 2 | "Suggested friends based on common interest:" | "Bob"   | "David" | "Music"        |

Started streaming 2 records after 45 ms and completed after 119 ms.

## 8. Find Shortest Path between Alice and Charlie

```
MATCH path = shortestPath((a:Person {name: 'Alice'})-[:FRIEND_OF*]-(c:Person {name: 'Charlie'}))
RETURN 'Shortest path between Alice and Charlie:' AS Description, path;
```



| Description                                | path                                                                                                                                   |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| "Shortest path between Alice and Charlie:" | (:Person {name: "Alice", age: 30})-[:FRIEND_OF]->(:Person {name: "David", age: 28})-<[:FRIEND_OF]-(:Person {name: "Charlie", age: 35}) |

## 9. Find Degree of Connection

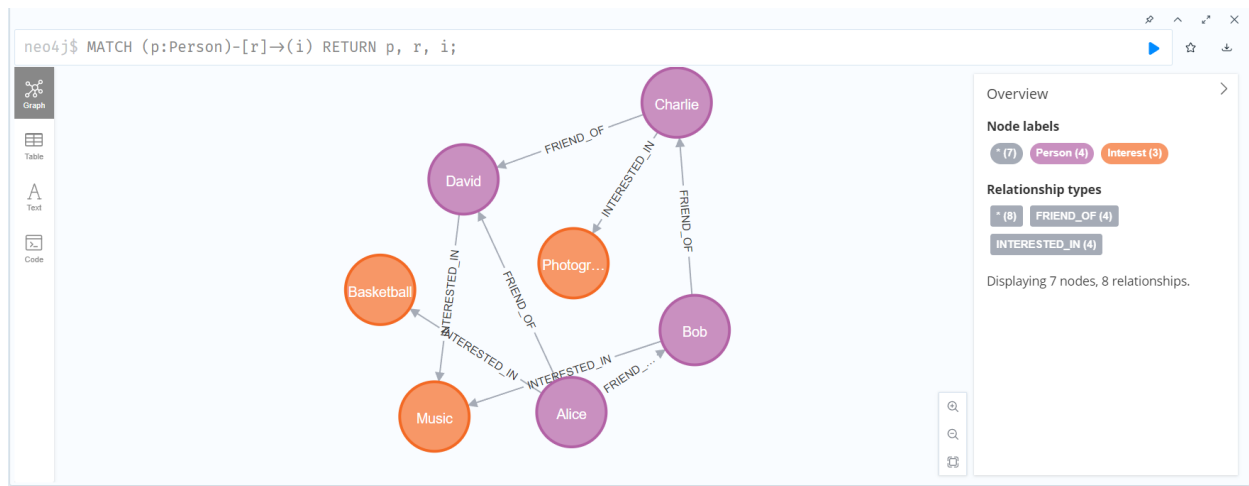
```
MATCH (p:Person)-[:FRIEND_OF]-(friends)
RETURN p.name AS Person, COUNT(friends) AS FriendCount;
```

```
neo4j$ MATCH (p:Person)-[:FRIEND_OF]-(friends) RETURN p.name AS Person, COUNT(friends) AS FriendCount;
```

| Person    | FriendCount |
|-----------|-------------|
| "Alice"   | 2           |
| "Bob"     | 2           |
| "Charlie" | 2           |
| "David"   | 2           |

## 10. Return the Graph

```
MATCH (p:Person)-[r]→(i)
RETURN p, r, i;
```



## 11. Delete all relations connected to Music

```
MATCH (:Interest {name: 'Music'})←[r]-(:Person)
DELETE r;
```

```
neo4j$ MATCH (:Interest {name: 'Music'})←[r]-(:Person) DELETE r;
```

Deleted 2 relationships, completed after 49 ms.

## 12. Delete relationship between Charlie and Photography

```
MATCH (n:Person {name: 'Charlie'})-[r:INTERESTED_IN]→()
DELETE r;
```

```
neo4j$ MATCH (n:Person {name: 'Charlie'})-[r:INTERESTED_IN]→() DELETE r;
```

Deleted 1 relationship, completed after 21 ms.

## Complex part

1. Deleting all previous nodes and relationships, creating a new ones

```
neo4j$ MATCH (n) DETACH DELETE n
```



Table



Deleted 7 nodes, deleted 5 relationships, completed after 18 ms.

```
1 // Create Suppliers
2 CREATE (s1:Supplier {name: 'Supplier 1', capacity: 1000})
3 CREATE (s2:Supplier {name: 'Supplier 2', capacity: 1200})
4 CREATE (s3:Supplier {name: 'Supplier 3', capacity: 800})
5
6 // Create Manufacturers
7 CREATE (m1:Manufacturer {name: 'Manufacturer 1', productionRate: 500})
8 CREATE (m2:Manufacturer {name: 'Manufacturer 2', productionRate: 300})
9 CREATE (m3:Manufacturer {name: 'Manufacturer 3', productionRate: 400})
10
11 // Create Warehouses/Distribution Centers
12 CREATE (w1:Warehouse {name: 'Warehouse 1', stock: 700})
```



Table



Code

Added 15 labels, created 15 nodes, set 84 properties, created 19 relationships, completed after 2883 ms.

## 2. Finding the shortest path by cost from Supplier to Retailer

```
1 MATCH (s:Supplier)-[rels*]-(r:Retailer)
2 WITH s, r, reduce(totalCost = 0, rel in rels | totalCost + rel.cost) AS totalCost
3 RETURN s.name AS Supplier, r.name AS Retailer, totalCost
4 ORDER BY totalCost ASC
5 LIMIT 1;
6
```



Table



Text



Code

|   | Supplier     | Retailer     | totalCost |
|---|--------------|--------------|-----------|
| 1 | "Supplier 1" | "Retailer 1" | 43        |

## 3. Finding the longest path by cost from Supplier to Retailer

```
1 MATCH (s:Supplier)-[rels*]-(r:Retailer)
2 WITH s, r, reduce(totalCost = 0, rel in rels | totalCost + rel.cost) AS totalCost
3 RETURN s.name AS Supplier, r.name AS Retailer, totalCost
4 ORDER BY totalCost DESC
5 LIMIT 1;
6
```



Table



Text



Code

|   | Supplier     | Retailer     | totalCost |
|---|--------------|--------------|-----------|
| 1 | "Supplier 3" | "Retailer 3" | 138       |

## 4. Finding the longest physical path (most hops) from Supplier to Retailer

```

1 MATCH p=(s:Supplier)-[rels*]→(r:Retailer)
2 RETURN s.name AS Supplier, r.name AS Retailer, length(p) AS pathLength
3 ORDER BY pathLength DESC
4 LIMIT 1;
5

```

|   | Supplier     | Retailer     | pathLength |
|---|--------------|--------------|------------|
| 1 | "Supplier 1" | "Retailer 1" | 4          |

## 5. Finding the shortest physical path (fewest hops) from Supplier to Retailer

```

1 MATCH p=(s:Supplier)-[rels*]→(r:Retailer)
2 RETURN s.name AS Supplier, r.name AS Retailer, length(p) AS pathLength
3 ORDER BY pathLength ASC
4 LIMIT 1;
5

```

|   | Supplier     | Retailer     | pathLength |
|---|--------------|--------------|------------|
| 1 | "Supplier 1" | "Retailer 1" | 3          |

## 6. Finding the farthest path by time from Supplier to Retailer

```

1 MATCH (s:Supplier)-[rels*]→(r:Retailer)
2 WITH s, r, reduce(totalCost = 0, rel in rels | totalCost + rel.cost) AS totalCost
3 RETURN s.name AS Supplier, r.name AS Retailer, totalCost
4 ORDER BY totalCost DESC
5 LIMIT 1;
6

```

|   | Supplier     | Retailer     | totalCost |
|---|--------------|--------------|-----------|
| 1 | "Supplier 3" | "Retailer 3" | 138       |

## 7. Finding the fastest route by time from Supplier to Retailer

```

1 MATCH (s:Supplier)-[rels*]→(r:Retailer)
2 WITH s, r, reduce(totalTime = 0, rel in rels | totalTime + rel.time) AS totalTime
3 RETURN s.name AS Supplier, r.name AS Retailer, totalTime
4 ORDER BY totalTime ASC
5 LIMIT 1;
6

```

|   | Supplier     | Retailer     | totalTime |
|---|--------------|--------------|-----------|
| 1 | "Supplier 1" | "Retailer 1" | 10        |

## 8. Creating a new faster path relation

```

1 MATCH (s:Supplier {name: 'Supplier 1'}), (r:Retailer {name: 'Retailer 1'})
2 CREATE (s)-[:SUPPLIES {mode: 'express', cost: 15, time: 1}]->(r);
3

```

Set 3 properties, created 1 relationship, completed after 35 ms.

## 9. Finding the fastest path by time from Supplier to Retailer

```
1 MATCH (s:Supplier)-[rels*]→(r:Retailer)
2 WITH s, r, reduce(totalTime = 0, rel in rels | totalTime + rel.time) AS totalTime
3 RETURN s.name AS Supplier, r.name AS Retailer, totalTime
4 ORDER BY totalTime ASC
5 LIMIT 1;
6
```

|       | Supplier     | Retailer     | totalTime |
|-------|--------------|--------------|-----------|
| Table | "Supplier 1" | "Retailer 1" | 1         |
| Text  |              |              |           |
|       |              |              |           |